Programming with Julia – CHEATSHEET

Sean McGowan

Getting started

JULIA-ENVIRONMENT

Interface

- REPL: execute code
- Editor: editing scripts/notebooks

Useful commands

- Check version: `versioninfo()`
- List objects: `names(Main)`
- Clear REPL: `<ctrl> + L`

FILES

- Run script: in VSCode press `Run`
- Run selected lines: `<shift> + <enter>`
- Comment code: `# a comment`
- Code cell: `#%% a code cell`

Comments explain *why* not *what*
Unicode support: $\sqrt{2}$, α, $F_{x,y}$

Objects

TYPES

- Numeric, logical, character
- Check type: `typeof(x)`

Assign variables: `x = 1`, `y = true`, `z = "a"` **Suppress output:** `x = 1;`

COMMON OBJECTS

- Strings: "Hello world!"
- Vectors: `[1,2,3]`
- Matrices: `[8 7 6 5; 4 3 2 1]`
- Tuples: `(1,"abc",[1,2,3])`
- Named tuples: `(x = [1,2,3],y = "Hello")`

Modify and copy objects:
`push!()`, `append!()`, `copy()`

OPERATORS

- Arithmetic: `+ - * / ^`
- Updating: `+= -= *= /=`
- Logical: `< > <= >=`, `&&` (and), `||` (or), `==` (equals), `!=` (not equals)

INDEXING

- Starts at 1
- Element: `x[i]`
- Range: `x[start:step:stop]`, starting at `start`, stopping at `stop`, incrementing by `step`
- Tuple unpacking: `a,b,c = T`
- Named tuples: `T.name`, `T[:name]`

Control flow

STRUCTURES

If / else: branched decision making

```
if x < 0
  ...
  elseif x == 0
  ...
  else:
  ...
end
```

For: iterate over sequences

```
for i in 1:5
  ...
end
```

While: loop until condition is false

```
while x < 1
  ...
end
```

Always *preallocate* objects before loops

Nested loops: loops within loops

```
for i in 1:4, j in 1:4:
    if i*j = 4:
      ...
    end
end
```

Loop control

- `break`: stop loop
- `continue`: skip iteration

List comprehension: list from existing list

```
[expression for item in sequence
  if condition == True]
```

Functions

DEFINING AND USING FUNCTIONS

Inline: `f(x,y) = x+2y`

Function block: longer, many-lined functions

```
function f(x,y)
  return x+2y
end
```

Anonymous functions: no-name functions within functions

```
x -> x^2
```

Multiple outputs: return a tuple

Defaults: arguments with default values

```
g(a,b;n=2) = a^n - b^n
```

Apply funcion element-wise:
`broadcast(function,X)`, `function.(X)`

Variable scope: local, global

Reading and writing files:
read ('r'), write ('w'), append ('a')

```
open("file", "r") do f
    ...
end
```

USEFUL FUNCTIONS

- `round()`, `floor()`, `ceil()`
- `abs()`, `sign()`
- `sqrt()`, `cbrt()`, `exp()`, `log()`, `log10()`
- `sin()`, `cos()`, `tan()`, `sinh()`, `cosh()`, `tanh()`, `asin()`, `acos()`, `atan()`

Packages

USING PACKAGES

- Install: `using Pkg; Pkg.add("Plots")`
- Load: `using Plots`

POPULAR PACKAGES

- Plotting: `Plots`, `Gadfly`, `PyPlot`
- Math: `Differential Equations`, `Symbolics`, `Calculus`
- Stats: `StatsBase`, `Distributions`

Debugging and help

Common errors

- BoundsError: index out of range
- MethodError: wrong argument type
- DivideError: divide by zero
- UndefVarError: undefined symbol
- DimensionMismatch: incompatible array shapes

Debugging tools

- Error message and stack trace
- `Debugger` package

Documentation and other resources

- `?function/?package`: view documentation for functions and packages
- Online resources: official Julia tutorials, Stack Overflow, Julia YouTube channel